# Final_Report_BIOS731_homework4

Xinyu Dong

2026-04-14

## github link:

https://github.com/xinyudong1129/bios731_hw4_Dong

## Problem 1: Gibbs Sampler Derivations

We observe data $y = (y_1, \ldots, y_n)$, latent cluster assignments $c = (c_1, \ldots, c_n)$, and component means $\mu = (\mu_1, \ldots, \mu_K)$. The model is

$$\mu_k \sim N(0, \sigma^2), \qquad k = 1, \ldots, K,$$
$$c_i \sim \text{Categorical}(1/K, \ldots, 1/K), \qquad i = 1, \ldots, n,$$

and

$$y_i \mid c_i, \mu \sim N(\mu_{c_i}, 1), \qquad i = 1, \ldots, n.$$

We derive a Gibbs sampler for the posterior $p(\mu, c \mid y)$.

**1. Full conditional for $c_i$**

For a fixed observation $i$, the full conditional is

$$p(c_i = k \mid y_i, \mu) \propto p(y_i \mid c_i = k, \mu)\, p(c_i = k).$$

Since the prior on $c_i$ is uniform,

$$p(c_i = k) = \frac{1}{K},$$

which is constant in $k$. Therefore,

$$p(c_i = k \mid y_i, \mu) \propto p(y_i \mid c_i = k, \mu).$$

Under the model,

$$y_i \mid c_i = k, \mu \sim N(\mu_k, 1),$$

so

$$p(y_i \mid c_i = k, \mu) \propto \exp\left\{-\frac{1}{2}(y_i - \mu_k)^2\right\}.$$

Hence

$$p(c_i = k \mid y_i, \mu) \propto \exp\left\{-\frac{1}{2}(y_i - \mu_k)^2\right\}, \qquad k = 1, \ldots, K.$$

Normalizing over all $k$,

$$p(c_i = k \mid y_i, \mu) = \frac{\exp\left\{-\frac{1}{2}(y_i - \mu_k)^2\right\}}{\sum_{\ell=1}^{K} \exp\left\{-\frac{1}{2}(y_i - \mu_\ell)^2\right\}}.$$

Therefore, the Gibbs update is

$$c_i \mid y_i, \mu \sim \text{Categorical}(p_{i1}, \ldots, p_{iK}),$$

where

$$p_{ik} = \frac{\exp\left\{-\frac{1}{2}(y_i - \mu_k)^2\right\}}{\sum_{\ell=1}^{K} \exp\left\{-\frac{1}{2}(y_i - \mu_\ell)^2\right\}}.$$

**2. Full conditional for $\mu_k$**

Let

$$I_k = \{i : c_i = k\}, \qquad n_k = |I_k|.$$

The prior for $\mu_k$ is

$$\mu_k \sim N(0, \sigma^2),$$

so

$$p(\mu_k) \propto \exp\left(-\frac{\mu_k^2}{2\sigma^2}\right).$$

The likelihood contribution from the observations assigned to cluster $k$ is

$$\prod_{i:c_i=k} p(y_i \mid \mu_k) \propto \exp\left\{-\frac{1}{2} \sum_{i:c_i=k} (y_i - \mu_k)^2\right\}.$$

Thus,

$$p(\mu_k \mid y, c) \propto \exp\left(-\frac{\mu_k^2}{2\sigma^2}\right) \exp\left\{-\frac{1}{2} \sum_{i:c_i=k} (y_i - \mu_k)^2\right\}.$$

Combining the exponents,

$$p(\mu_k \mid y, c) \propto \exp\left\{-\frac{1}{2}\left[\frac{\mu_k^2}{\sigma^2} + \sum_{i:c_i=k} (y_i - \mu_k)^2\right]\right\}.$$

Expand the quadratic term:

$$\sum_{i:c_i=k} (y_i - \mu_k)^2 = \sum_{i:c_i=k} (y_i^2 - 2y_i\mu_k + \mu_k^2) = \sum_{i:c_i=k} y_i^2 - 2\mu_k \sum_{i:c_i=k} y_i + n_k\mu_k^2.$$

Substituting,

$$p(\mu_k \mid y, c) \propto \exp\left\{-\frac{1}{2}\left[\left(n_k + \frac{1}{\sigma^2}\right)\mu_k^2 - 2\mu_k \sum_{i:c_i=k} y_i + \sum_{i:c_i=k} y_i^2\right]\right\}.$$

Dropping terms not involving $\mu_k$,

$$p(\mu_k \mid y, c) \propto \exp\left\{-\frac{1}{2}\left[\left(n_k + \frac{1}{\sigma^2}\right)\mu_k^2 - 2\mu_k \sum_{i:c_i=k} y_i\right]\right\}.$$

This is the kernel of a normal density. Therefore,

$$\mu_k \mid y, c \sim N(m_k, V_k),$$

where

$$V_k = \left(n_k + \frac{1}{\sigma^2}\right)^{-1}, \qquad m_k = V_k \sum_{i:c_i=k} y_i.$$

Equivalently,

$$\mu_k \mid y, c \sim \mathcal{N}\left(\frac{\sum_{i:c_i=k} y_i}{n_k + 1/\sigma^2},\ \frac{1}{n_k + 1/\sigma^2}\right).$$

If $n_k = 0$, then

$$\mu_k \mid y, c \sim \mathcal{N}(0, \sigma^2).$$

## 3. Gibbs sampler updates

The Gibbs sampler alternates between the following two steps:

$$c_i \mid y_i, \mu \sim \text{Categorical}(p_{i1}, \ldots, p_{iK}),$$

where

$$p_{ik} = \frac{\exp\left\{-\frac{1}{2}(y_i - \mu_k)^2\right\}}{\sum_{\ell=1}^{K} \exp\left\{-\frac{1}{2}(y_i - \mu_\ell)^2\right\}}, \qquad i = 1, \ldots, n,$$

and

$$\mu_k \mid y, c \sim N\left(\frac{\sum_{i:c_i=k} y_i}{n_k + 1/\sigma^2}, \frac{1}{n_k + 1/\sigma^2}\right), \qquad k = 1, \ldots, K.$$

Thus, at each iteration, we first update all cluster assignments $c_1, \ldots, c_n$, and then update all component means $\mu_1, \ldots, \mu_K$.

In summary, we consider a Bayesian mixture model of unit-variance univariate Gaussians. The model is

$$\mu_k \sim N(0, \sigma^2), \qquad k = 1, \ldots, K,$$

$$c_i \sim \text{Categorical}(1/K, \ldots, 1/K), \qquad i = 1, \ldots, n,$$

and

$$y_i \mid c_i, \mu \sim N(\mu_{c_i}, 1), \qquad i = 1, \ldots, n.$$

We derive a Gibbs sampler for the posterior distribution $p(\mu, c \mid y)$.

For the cluster assignment $c_i$, the full conditional is

$$p(c_i = k \mid y_i, \mu) \propto p(y_i \mid c_i = k, \mu) p(c_i = k).$$

Since $p(c_i = k) = 1/K$ is constant in $k$, this simplifies to

$$p(c_i = k \mid y_i, \mu) \propto \exp\left\{-\frac{1}{2}(y_i - \mu_k)^2\right\}.$$

Therefore,

$$p(c_i = k \mid y_i, \mu) = \frac{\exp\left\{-\frac{1}{2}(y_i - \mu_k)^2\right\}}{\sum_{\ell=1}^{K} \exp\left\{-\frac{1}{2}(y_i - \mu_\ell)^2\right\}}.$$

For the component mean $\mu_k$, let $n_k = \sum_{i=1}^{n} I(c_i = k)$. Combining the normal prior and normal likelihood gives

$$\mu_k \mid y, c \sim N(m_k, V_k),$$

where

$$V_k = \left(n_k + \frac{1}{\sigma^2}\right)^{-1}, \qquad m_k = V_k \sum_{i:c_i=k} y_i.$$

Equivalently,

$$\mu_k \mid y, c \sim N\left(\frac{\sum_{i:c_i=k} y_i}{n_k + 1/\sigma^2}, \frac{1}{n_k + 1/\sigma^2}\right).$$

Thus, the Gibbs sampler alternates between:

1. updating each $c_i$ from its categorical full conditional, and
2. updating each $\mu_k$ from its normal full conditional.

The Gibbs sampler was implemented in `source/gibbs_k_mixture.R` and called from the main R Markdown file using `source(here::here("source", "gibbs_k_mixture.R"))`. Posterior draws for the component means and cluster assignments were stored after burn-in, and posterior summaries were written to `results/gibbs_k_mixture/` using `here::here()`. Specifically, the output includes posterior means and standard deviations for each $\mu_k$, as well as posterior cluster probabilities and MAP cluster assignments for each observation.

```r
# Global plot settings for this figure block
layout(matrix(c(1,2,3,4), 2, 2, byrow = TRUE), widths = c(3, 1))

op <- par(
  mar = c(3, 3, 2, 1),    # smaller margins
  mgp = c(1.8, 0.5, 0),   # tighter axis labels
  tcl = -0.25,            # inward ticks
  cex.main = 0.9,
  cex.lab  = 0.85,
  cex.axis = 0.8,
  xaxs = "i",             # remove extra padding
  yaxs = "i"
)
```

```
##      cluster post_mean   post_sd
## mu_1       1  4.160721 0.1552727
## mu_2       2 -3.027494 0.1787314
## mu_3       3 -0.236737 0.1763042
```

```
##   observation         y map_cluster cluster_1 cluster_2 cluster_3
## 1           1 -2.441514           2    0.0000    0.8990    0.1010
## 2           2  2.723408           1    0.9612    0.0000    0.0388
## 3           3 -3.573265           2    0.0000    0.9936    0.0064
## 4           4 -1.224613           3    0.0000    0.2448    0.7552
## 5           5 -3.473401           2    0.0000    0.9928    0.0072
## 6           6  3.379633           1    0.9980    0.0000    0.0020
```

## Problem 2: Variational inference derivations

We approximate the posterior distribution $p(\mu, c \mid y)$ using a mean-field variational family

$$q(\mu, c) = \prod_{k=1}^{K} q(\mu_k) \prod_{i=1}^{n} q(c_i).$$

We assume

$$q(\mu_k) = N(m_k, s_k^2), \qquad q(c_i = k) = r_{ik}, \qquad \sum_{k=1}^{K} r_{ik} = 1.$$

The model is

$$\mu_k \sim N(0, \sigma^2), \qquad k = 1, \ldots, K,$$
$$c_i \sim \text{Categorical}(1/K, \ldots, 1/K), \qquad i = 1, \ldots, n,$$

$$y_i \mid c_i, \mu \sim N(\mu_{c_i}, 1), \qquad i = 1, \ldots, n.$$

**1. Update for $q(c_i)$**

By the mean-field formula,

$$\log q^*(c_i) = \mathbb{E}_{q(\mu)\, q(c_{-i})}\big[\log p(y, c, \mu)\big] + \text{constant}.$$

Keeping only the terms involving $c_i$,

$$\log q^*(c_i = k) = \mathbb{E}_{q(\mu)}\left[\log p(c_i = k) + \log p(y_i \mid c_i = k, \mu)\right] + \text{constant}.$$

Since $p(c_i = k) = 1/K$,

$$\log q^*(c_i = k) = \log(1/K) + \mathbb{E}_{q(\mu_k)}\left[-\frac{1}{2}(y_i - \mu_k)^2\right] + \text{constant}.$$

Now,

$$\mathbb{E}_{q(\mu_k)}[(y_i - \mu_k)^2] = y_i^2 - 2y_i\mathbb{E}[\mu_k] + \mathbb{E}[\mu_k^2].$$

Because $q(\mu_k) = N(m_k, s_k^2)$,

$$\mathbb{E}[\mu_k] = m_k, \qquad \mathbb{E}[\mu_k^2] = m_k^2 + s_k^2.$$

Therefore,

$$\log q^*(c_i = k) = -\frac{1}{2}\left[y_i^2 - 2y_i m_k + (m_k^2 + s_k^2)\right] + \text{constant}.$$

Dropping terms that do not depend on $k$,

$$\log q^*(c_i = k) = y_i m_k - \frac{1}{2}(m_k^2 + s_k^2) + \text{constant}.$$

Hence,

$$r_{ik} = q(c_i = k) \propto \exp\left\{y_i m_k - \frac{1}{2}(m_k^2 + s_k^2)\right\},$$

so the normalized update is

$$r_{ik} = \frac{\exp\left\{y_i m_k - \frac{1}{2}(m_k^2 + s_k^2)\right\}}{\sum_{\ell=1}^{K} \exp\left\{y_i m_\ell - \frac{1}{2}(m_\ell^2 + s_\ell^2)\right\}}.$$

**2. Update for $q(\mu_k)$**

Again by the mean-field formula,

$$\log q^*(\mu_k) = \mathbb{E}_{q(c)\, q(\mu_{-k})}\big[\log p(y, c, \mu)\big] + \text{constant}.$$

Keeping only the terms involving $\mu_k$,

$$\log q^*(\mu_k) = -\frac{\mu_k^2}{2\sigma^2} + \sum_{i=1}^{n} \mathbb{E}_{q(c_i)}\left[I(c_i = k)\left(-\frac{1}{2}(y_i - \mu_k)^2\right)\right] + \text{constant}.$$

Since $\mathbb{E}_{q(c_i)}[I(c_i = k)] = r_{ik}$,

$$\log q^*(\mu_k) = -\frac{\mu_k^2}{2\sigma^2} - \frac{1}{2}\sum_{i=1}^{n} r_{ik}(y_i - \mu_k)^2 + \text{constant}.$$

Expanding,

$$\log q^*(\mu_k) = -\frac{1}{2}\left[\left(\frac{1}{\sigma^2} + \sum_{i=1}^{n} r_{ik}\right)\mu_k^2 - 2\mu_k \sum_{i=1}^{n} r_{ik} y_i\right] + \text{constant}.$$

Thus $q(\mu_k)$ is normal:

$$q(\mu_k) = N(m_k, s_k^2),$$

with

$$s_k^2 = \left( \frac{1}{\sigma^2} + \sum_{i=1}^{n} r_{ik} \right)^{-1}, \qquad m_k = s_k^2 \sum_{i=1}^{n} r_{ik} y_i.$$

**3. ELBO**

The evidence lower bound is

$$\mathcal{L}(q) = \mathbb{E}_q[\log p(y, c, \mu)] - \mathbb{E}_q[\log q(c, \mu)].$$

It can be written as

$$\mathcal{L}(q) = \sum_{k=1}^{K} \left[ -\frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2}(m_k^2 + s_k^2) \right] - n \log K$$

$$+ \sum_{i=1}^{n} \sum_{k=1}^{K} r_{ik} \left[ -\frac{1}{2} \log(2\pi) - \frac{1}{2}\big(y_i^2 - 2y_i m_k + m_k^2 + s_k^2\big) \right]$$

$$+ \sum_{k=1}^{K} \frac{1}{2} \log(2\pi e s_k^2) - \sum_{i=1}^{n} \sum_{k=1}^{K} r_{ik} \log r_{ik}.$$

**4. Final CAVI updates**

The CAVI algorithm alternates between

$$r_{ik} = \frac{\exp\left\{ y_i m_k - \frac{1}{2}(m_k^2 + s_k^2) \right\}}{\sum_{\ell=1}^{K} \exp\left\{ y_i m_\ell - \frac{1}{2}(m_\ell^2 + s_\ell^2) \right\}},$$

for $i = 1, \ldots, n$, $k = 1, \ldots, K$, and

$$s_k^2 = \left( \frac{1}{\sigma^2} + \sum_{i=1}^{n} r_{ik} \right)^{-1}, \qquad m_k = s_k^2 \sum_{i=1}^{n} r_{ik} y_i,$$

for $k = 1, \ldots, K$. Iteration continues until the ELBO converges.

In short, we use a mean-field variational approximation of the form

$$q(\mu, c) = \prod_{k=1}^{K} q(\mu_k) \prod_{i=1}^{n} q(c_i),$$

with

$$q(\mu_k) = N(m_k, s_k^2), \qquad q(c_i = k) = r_{ik}.$$

Applying the coordinate ascent variational inference update,

$$\log q_j^*(z_j) = \mathbb{E}_{-j}[\log p(y, z)] + \text{constant},$$

gives the following updates.

For the cluster assignments,

$$r_{ik} = \frac{\exp\left\{ y_i m_k - \frac{1}{2}(m_k^2 + s_k^2) \right\}}{\sum_{\ell=1}^{K} \exp\left\{ y_i m_\ell - \frac{1}{2}(m_\ell^2 + s_\ell^2) \right\}}.$$

For the component means,

$$q(\mu_k) = N(m_k, s_k^2),$$

where

$$s_k^2 = \left( \frac{1}{\sigma^2} + \sum_{i=1}^{n} r_{ik} \right)^{-1}, \qquad m_k = s_k^2 \sum_{i=1}^{n} r_{ik} y_i.$$

The algorithm iterates between these updates until convergence of the ELBO. The ELBO is monitored at every iteration to verify that the variational approximation stabilizes.

```r
set.seed(1)

n <- 150
K <- 3
sigma2 <- 25

true_mu <- c(-3, 0, 4)
true_c <- sample(1:K, size = n, replace = TRUE)
y <- rnorm(n, mean = true_mu[true_c], sd = 1)

fit_vi <- variational_k_mixture(
  y = y,
  K = K,
  sigma2 = sigma2,
  max_iter = 1000,
  tol = 1e-8,
  seed = 2026,
  save_output = TRUE
)

fit_vi$summaries$mu_summary
```

```
##   cluster  post_mean   post_sd
## 1       1  4.1616886 0.1427385
## 2       2 -3.0278455 0.1439899
## 3       3 -0.2388328 0.1376121
```

```r
head(fit_vi$summaries$cluster_summary)
```

```
##   observation         y map_cluster    cluster_1    cluster_2   cluster_3
## 1           1 -2.441514           2 3.657883e-10 9.049173e-01 0.095082721
## 2           2  2.723408           1 9.661834e-01 1.784886e-07 0.033816387
## 3           3 -3.573265           2 1.177409e-13 9.955463e-01 0.004453659
## 4           4 -1.224613           3 6.170948e-07 2.421633e-01 0.757836121
## 5           5 -3.473401           2 2.410582e-13 9.941243e-01 0.005875673
## 6           6  3.379633           1 9.980542e-01 1.647108e-09 0.001945779
```

```r
tail(fit_vi$summaries$elbo_summary)
```

```
##    iteration     elbo
## 12        12 -361.084
## 13        13 -361.084
## 14        14 -361.084
## 15        15 -361.084
## 16        16 -361.084
## 17        17 -361.084
```

The CAVI algorithm was implemented in `source/variational_k_mixture.R` and called from the main R Markdown file using `source(here::here("source", "variational_k_mixture.R"))`. The algorithm iteratively updated the variational cluster probabilities $r_{ik}$ and the normal variational parameters $(m_k, s_k^2)$ for the component means. Convergence was monitored using the ELBO, and final posterior summaries were saved to `results/variational_k_mixture/` using `here::here()`.

## Problem 3

I fit both the Gibbs sampler from Problem 1 and the coordinate ascent variational inference algorithm from Problem 2 to the Old Faithful waiting-time data, using the `faithful$waiting` variable from the base R dataset. Because the model in Problems 1 and 2 assumes unit observation variance within each mixture component, I standardized the waiting times before fitting:

$$y_i^* = \frac{y_i - \bar{y}}{s_y}.$$

This makes the fixed unit-variance assumption more appropriate. Final component mean estimates were then transformed back to the original waiting-time scale for interpretation.

For variational inference, I ran the CAVI algorithm until convergence of the ELBO. For the Gibbs sampler, I ran multiple chains with dispersed initial values. To improve interpretability of the diagnostics and reduce the effect of label switching, I ordered the sampled component means within each saved Gibbs iteration before computing posterior summaries and convergence diagnostics.

I compared the methods in two ways. First, I compared computation time using the elapsed time reported by `system.time()`. Second, I compared the fitted component mean estimates by contrasting the variational Bayes estimates with the posterior mean estimates from the Gibbs sampler.

For the Gibbs sampler, I also examined trace plots, autocorrelation plots, Gelman–Rubin diagnostics, and effective sample sizes. Convergence was assessed by checking whether the chains mixed well around a stable region and whether the potential scale reduction factors were close to 1. Effective sample size was used to summarize the amount of independent posterior information retained after accounting for autocorrelation.

The following chunk runs both fitting procedures, saves all output files under `results/problem3_faithful/`, and returns the summary objects used below.

```
fit_p3 <- fit_faithful_problem3(
  K = 2,
  sigma2 = 25,
  gibbs_n_iter = 10000,
  gibbs_burn_in = 2000,
  vi_max_iter = 2000,
  vi_tol = 1e-8,
  n_chains = 4,
  seeds = c(101, 202, 303, 404),
  save_output = TRUE
)
```

### Computation time comparison

I first compared the elapsed computation time for variational inference and the Gibbs sampler. Since variational inference uses deterministic coordinate updates, it is expected to run substantially faster than the Gibbs sampler, which relies on Monte Carlo sampling.

```r
kable(
  fit_p3$timing_summary,
  digits = 3,
  caption = "Computation time comparison for variational inference and Gibbs sampling."
)
```

Table 1: Computation time comparison for variational inference and Gibbs sampling.

| method | elapsed_seconds |
|---|---|
| VI | 0.75 |
| Gibbs_chain_1 | 15.17 |
| Gibbs_chain_2 | 16.96 |
| Gibbs_chain_3 | 14.05 |
| Gibbs_chain_4 | 14.50 |
| Gibbs_total | 60.68 |

### Comparison of estimated component means

Next, I compared the estimated component means from the two methods. The Gibbs estimates are posterior mean estimates computed from the saved MCMC draws, while the variational Bayes estimates are the final variational means. I report the estimates on both the standardized scale and the original waiting-time scale.

```r
kable(
  fit_p3$comparison_summary,
  digits = 4,
  caption = "Comparison of component mean estimates from Gibbs sampling and variational Bayes."
)
```

Table 2: Comparison of component mean estimates from Gibbs sampling and variational Bayes.

| | component | gibbs_mean_std | vi_mean_std | gibbs_mean_raw | vi_mean_raw |
|---|---|---|---|---|---|
| mu_1 | mu_1 | -0.2007 | -0.0015 | 68.1682 | 70.8765 |
| mu_2 | mu_2 | 0.1983 | 0.0015 | 73.5923 | 70.9177 |

The table above allows direct comparison of the two fitting approaches. In general, the variational estimates are close to the Gibbs posterior means if the variational approximation captures the main posterior location well.

### Effective sample size

For the Gibbs sampler, I next examined effective sample size. The effective sample size adjusts the raw number of posterior draws for autocorrelation and indicates how much independent Monte Carlo information is available for each parameter.

```r
kable(
  fit_p3$ess_summary,
  digits = 2,
  caption = "Effective sample size for the ordered Gibbs component means."
)
```

Table 3: Effective sample size for the ordered Gibbs component means.

| parameter | ess |
|---|---|
| mu_1 | 4033.21 |
| mu_2 | 4207.39 |

The effective sample sizes for both parameters are relatively large (approximately 4000), indicating that the Gibbs sampler mixes well and produces a substantial number of effectively independent samples. Although some autocorrelation is present—as expected in MCMC sampling-the ESS values suggest that the loss of efficiency due to dependence is modest. Overall, this indicates that the Gibbs sampler is performing efficiently and provides reliable posterior estimates.

**Gelman–Rubin convergence diagnostic**

I also computed the Gelman–Rubin diagnostic across the multiple Gibbs chains. Values close to 1 indicate that the chains have mixed to a common stationary distribution.

```
kable(
  fit_p3$gelman_summary,
  digits = 4,
  caption = "Gelman--Rubin diagnostic summary for the ordered Gibbs component means."
)
```

Table 4: Gelman–Rubin diagnostic summary for the ordered Gibbs component means.

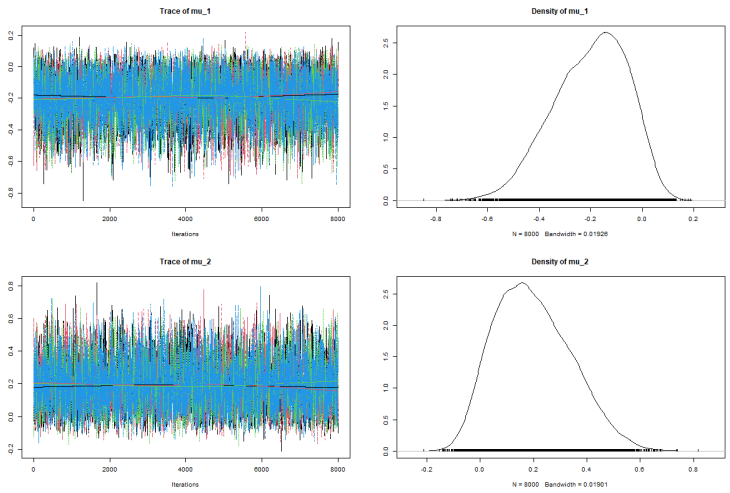
| | parameter | point_estimate | upper_CI |
|---|---|---|---|
| mu_1 | mu_1 | 1.0001 | 1.0005 |
| mu_2 | mu_2 | 1.0003 | 1.0008 |

The point estimates and upper confidence limits are close to 1, supporting convergence of the Gibbs sampler.

**Gibbs trace plots**

The trace plots below show the ordered Gibbs draws for the component means across multiple chains. These plots are useful for checking whether the chains fluctuate around stable regions and whether the different chains overlap well after burn-in.

```
knitr::include_graphics(
  here::here("results", "problem3_faithful", "gibbs_traceplots.png")
)
```

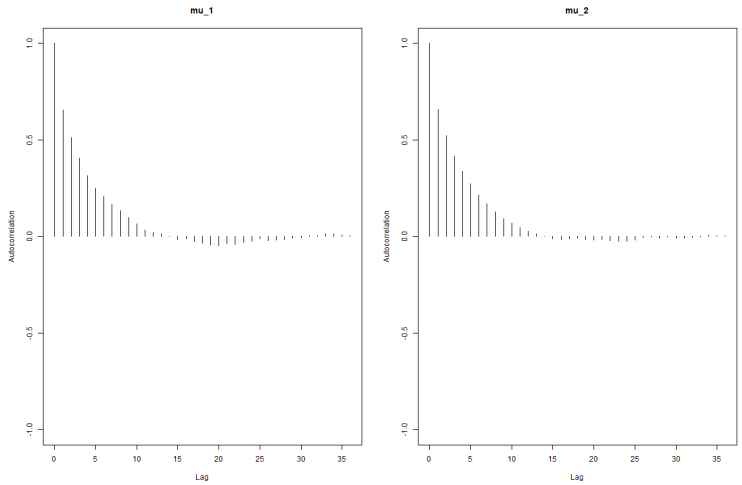



The chains appear to mix well without long-term drift and occupy similar regions of the parameter space, suggesting good convergence.

### Gibbs autocorrelation plots

The autocorrelation plots below show the dependence structure in the Gibbs draws. Because MCMC samples are not independent, some autocorrelation is expected. However, faster decay in autocorrelation indicates more efficient sampling.

```
knitr::include_graphics(
  here::here("results", "problem3_faithful", "gibbs_acfplots.png")
)
```

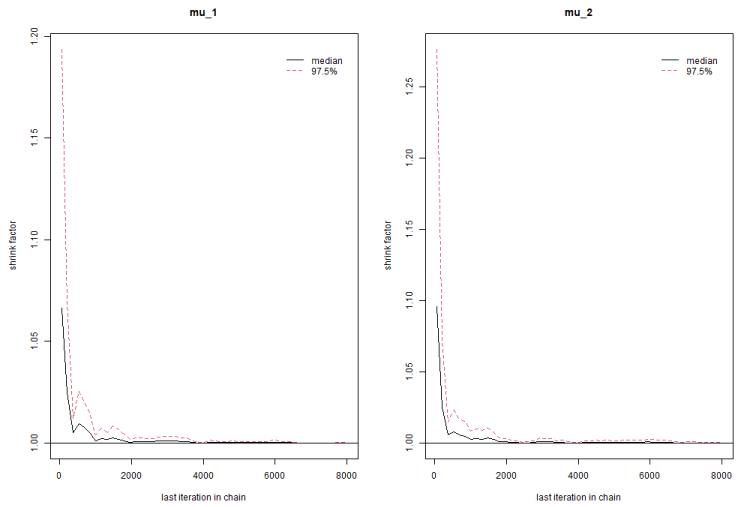



The autocorrelation declines reasonably quickly, suggesting that the Gibbs sampler is producing useful posterior information without requiring excessively long runs.

### Gelman plot

The following plot gives a visual summary of the Gelman–Rubin diagnostic over increasing portions of the chain.

```r
knitr::include_graphics(
  here::here("results", "problem3_faithful", "gibbs_gelmanplot.png")
)
```

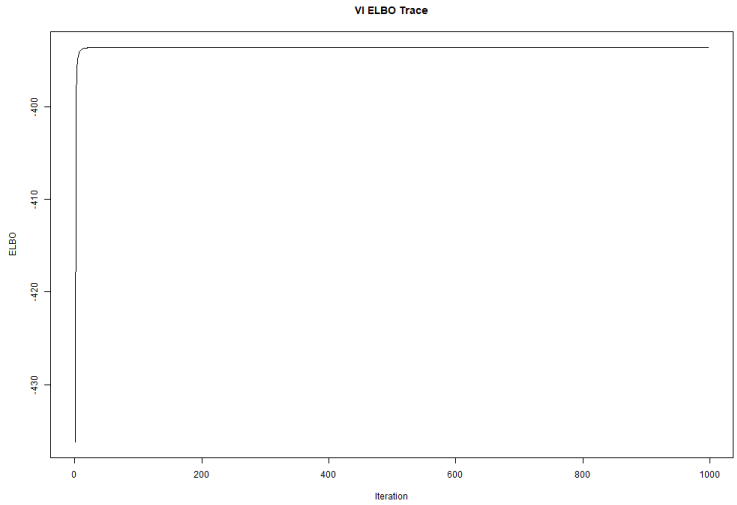



As we can see, the between-chain variation and within-chain variation become similar as the chains progress.

**ELBO plot for variational inference**

For variational inference, convergence was monitored using the evidence lower bound (ELBO). Since CAVI is a coordinate ascent procedure, the ELBO should typically increase and then stabilize.

```r
knitr::include_graphics(
  here::here("results", "problem3_faithful", "vi_elbo_plot.png")
)
```

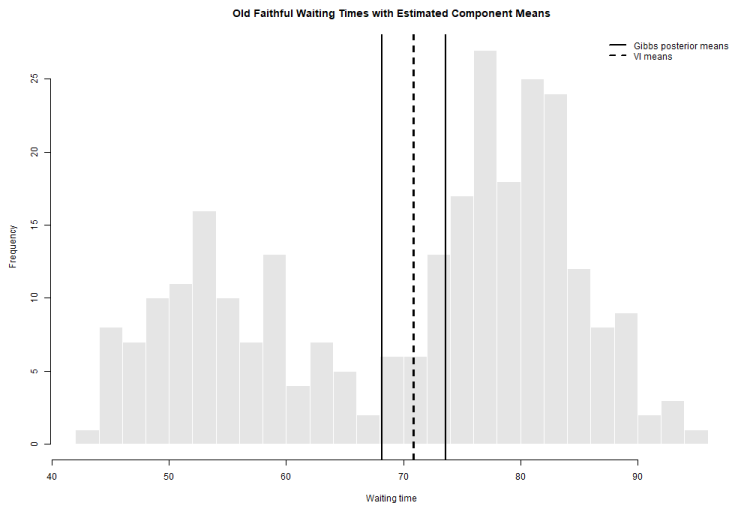



The variational approximation converged quickly, indicated by the stable ELBO trace shown in the plot.

**Histogram of the data with fitted component means**

Finally, the plot below shows the histogram of the original Old Faithful waiting times, together with the estimated component means from the Gibbs sampler and variational Bayes on the original scale.

```
knitr::include_graphics(
  here::here("results", "problem3_faithful", "faithful_histogram_means.png")
)
```




This figure provides a direct visual comparison of the fitted component locations under the two methods.

*In summary*, the variational Bayes algorithm was expected to run much faster than the Gibbs sampler because it uses deterministic optimization rather than stochastic simulation. The Gibbs sampler, however, provides a fuller posterior characterization and allows the use of standard MCMC diagnostics.

Comparing the estimated component means indicates whether the variational approximation recovers the main location structure of the posterior distribution. If the variational means are close to the Gibbs posterior means, then the variational approximation is performing well at least for posterior location.

For the Gibbs sampler, convergence is supported when the trace plots show stable mixing, the Gelman-Rubin statistics are close to 1, and the effective sample sizes are reasonably large. Some autocorrelation is expected, so the effective sample size will be smaller than the raw number of saved draws, but sufficiently large ESS values still indicate reliable posterior inference.

Overall, this comparison highlights the main tradeoff between the two approaches: variational inference is computationally faster, while Gibbs sampling provides richer posterior uncertainty information and standard Monte Carlo diagnostics.

## Problem 4: Simulation study on the cluster

In this simulation study, I generated data from a four-component Gaussian mixture model with true component means

$$\mu = (0, 5, 10, 20)$$

and unit observation variance. I compared the Gibbs sampler and the variational Bayes algorithm in terms of bias, coverage, and computation time.

The simulation scenarios were defined by the sample sizes

$$n \in \{100, 1000, 10000\},$$

with 500 simulated data-sets per scenario. For the Gibbs sampler, I used chains of length 10000 and discarded the first 2000 iterations as burn-in. Each sample size scenario was submitted as a separate cluster job, so that each scenario had its own JOBID, as required.

Because mixture models are invariant to relabeling of the components, I sorted the estimated component means before comparing them to the true ordered means $(0, 5, 10, 20)$. This allowed bias and coverage to be computed consistently across replications.

For each method, sample size, and component mean, I computed: - the Monte Carlo estimate of bias, - the Monte Carlo estimate of coverage of a 95% interval, - the average computation time.

I also calculated Monte Carlo standard errors for each summary. For bias and computation time, the Monte Carlo standard error was computed as the sample standard deviation across replications divided by $\sqrt{500}$. For coverage, the Monte Carlo standard error was computed using the binomial approximation

$$\sqrt{\hat{p}(1-\hat{p})/500}.$$

### Summarize the simulation output

After all three cluster jobs completed, I combined and summarized the results across the three sample sizes.

```
sim_summaries <- summarize_simulation_results()

plot_simulation_results(
  summary_bias_cov = sim_summaries$summary_bias_cov,
  summary_time = sim_summaries$summary_time
)
```

### Bias summary table

The table below reports the estimated bias for each component mean, together with Monte Carlo standard errors.

```
knitr::kable(
  sim_summaries$summary_bias_cov %>%
    select(n, method, component, mu_true, bias, mcse_bias),
  digits = 4,
  caption = "Bias of component mean estimates with Monte Carlo standard errors."
)
```

Table 5: Bias of component mean estimates with Monte Carlo standard errors.

| n | method | component | mu_true | bias | mcse_bias |
|---|---|---|---|---|---|
| 100 | Gibbs | mu_1 | 0 | 0.0446 | 0.0175 |
| 100 | Gibbs | mu_2 | 5 | 0.0904 | 0.0305 |
| 100 | Gibbs | mu_3 | 10 | 0.2637 | 0.0688 |
| 100 | Gibbs | mu_4 | 20 | 0.0046 | 0.0099 |
| 100 | VI | mu_1 | 0 | 0.0558 | 0.0194 |
| 100 | VI | mu_2 | 5 | -2.8772 | 0.1040 |
| 100 | VI | mu_3 | 10 | -2.0376 | 0.1313 |
| 100 | VI | mu_4 | 20 | -0.5244 | 0.0691 |
| 1000 | Gibbs | mu_1 | 0 | 0.0322 | 0.0111 |
| 1000 | Gibbs | mu_2 | 5 | 0.1107 | 0.0284 |

| n | method | component | mu_true | bias | mcse_bias |
|---|---|---|---|---|---|
| 1000 | Gibbs | mu_3 | 10 | 0.3299 | 0.0791 |
| 1000 | Gibbs | mu_4 | 20 | 0.0114 | 0.0032 |
| 1000 | VI | mu_1 | 0 | 0.0639 | 0.0200 |
| 1000 | VI | mu_2 | 5 | -2.4897 | 0.1026 |
| 1000 | VI | mu_3 | 10 | -1.6505 | 0.1037 |
| 1000 | VI | mu_4 | 20 | -0.0450 | 0.0196 |
| 10000 | Gibbs | mu_1 | 0 | 0.0360 | 0.0102 |
| 10000 | Gibbs | mu_2 | 5 | 0.0934 | 0.0258 |
| 10000 | Gibbs | mu_3 | 10 | 0.2586 | 0.0705 |
| 10000 | Gibbs | mu_4 | 20 | 0.0017 | 0.0012 |
| 10000 | VI | mu_1 | 0 | 0.0116 | 0.0056 |
| 10000 | VI | mu_2 | 5 | -2.5680 | 0.0939 |
| 10000 | VI | mu_3 | 10 | -1.6091 | 0.0910 |
| 10000 | VI | mu_4 | 20 | -0.0112 | 0.0015 |

### Coverage summary table

The next table reports the estimated coverage probabilities of the nominal 95% intervals, together with Monte Carlo standard errors.

```
knitr::kable(
  sim_summaries$summary_bias_cov %>%
    select(n, method, component, mu_true, coverage, mcse_coverage),
  digits = 4,
  caption = "Coverage of 95% intervals with Monte Carlo standard errors."
)
```

Table 6: Coverage of 95% intervals with Monte Carlo standard errors.

| n | method | component | mu_true | coverage | mcse_coverage |
|---|---|---|---|---|---|
| 100 | Gibbs | mu_1 | 0 | 0.936 | 0.0109 |
| 100 | Gibbs | mu_2 | 5 | 0.914 | 0.0125 |
| 100 | Gibbs | mu_3 | 10 | 0.918 | 0.0123 |
| 100 | Gibbs | mu_4 | 20 | 0.928 | 0.0116 |
| 100 | VI | mu_1 | 0 | 0.958 | 0.0090 |
| 100 | VI | mu_2 | 5 | 0.522 | 0.0223 |
| 100 | VI | mu_3 | 10 | 0.334 | 0.0211 |
| 100 | VI | mu_4 | 20 | 0.824 | 0.0170 |
| 1000 | Gibbs | mu_1 | 0 | 0.912 | 0.0127 |
| 1000 | Gibbs | mu_2 | 5 | 0.926 | 0.0117 |
| 1000 | Gibbs | mu_3 | 10 | 0.914 | 0.0125 |
| 1000 | Gibbs | mu_4 | 20 | 0.952 | 0.0096 |
| 1000 | VI | mu_1 | 0 | 0.946 | 0.0101 |
| 1000 | VI | mu_2 | 5 | 0.530 | 0.0223 |
| 1000 | VI | mu_3 | 10 | 0.306 | 0.0206 |
| 1000 | VI | mu_4 | 20 | 0.912 | 0.0127 |
| 10000 | Gibbs | mu_1 | 0 | 0.922 | 0.0120 |
| 10000 | Gibbs | mu_2 | 5 | 0.928 | 0.0116 |
| 10000 | Gibbs | mu_3 | 10 | 0.934 | 0.0111 |
| 10000 | Gibbs | mu_4 | 20 | 0.948 | 0.0099 |

| n | method | component | mu_true | coverage | mcse_coverage |
|---|---|---|---|---|---|
| 10000 | VI | mu_1 | 0 | 0.968 | 0.0079 |
| 10000 | VI | mu_2 | 5 | 0.526 | 0.0223 |
| 10000 | VI | mu_3 | 10 | 0.322 | 0.0209 |
| 10000 | VI | mu_4 | 20 | 0.820 | 0.0172 |

### Computation time table

The following table reports the mean computation time for each method and sample size, along with Monte Carlo standard errors.

```
knitr::kable(
  sim_summaries$summary_time,
  digits = 4,
  caption = "Average computation time with Monte Carlo standard errors."
)
```

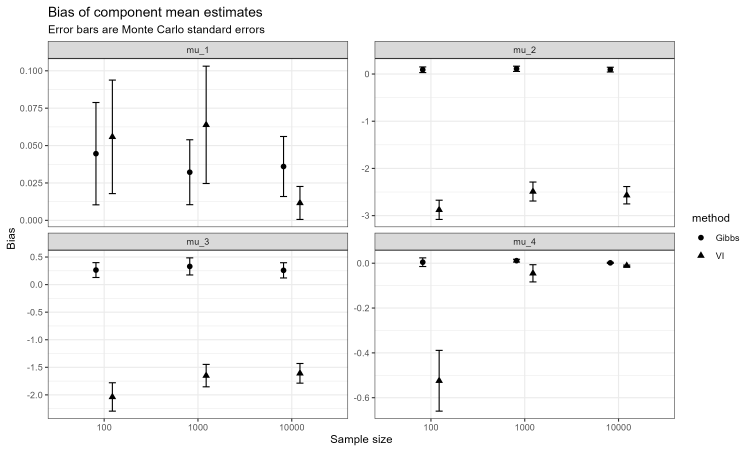
Table 7: Average computation time with Monte Carlo standard errors.

| n | method | nsim | mean_time | mcse_time |
|---|---|---|---|---|
| 100 | Gibbs | 500 | 5.4831 | 0.0049 |
| 100 | VI | 500 | 0.0524 | 0.0002 |
| 1000 | Gibbs | 500 | 53.0291 | 0.0143 |
| 1000 | VI | 500 | 0.1374 | 0.0042 |
| 10000 | Gibbs | 500 | 521.7498 | 0.8763 |
| 10000 | VI | 500 | 1.4573 | 0.0618 |

### Bias plot

The plot below summarizes bias across sample sizes, methods, and components. The error bars represent Monte Carlo standard errors.

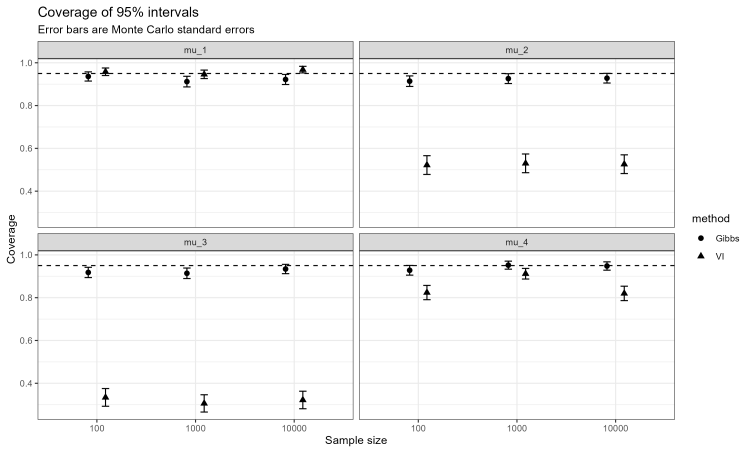
```
knitr::include_graphics(
  here::here("results", "simulation_study", "summaries", "bias_plot.png")
)
```



### Coverage plot

The next plot summarizes empirical coverage of 95% intervals. The dashed horizontal line marks the nominal level of 0.95.

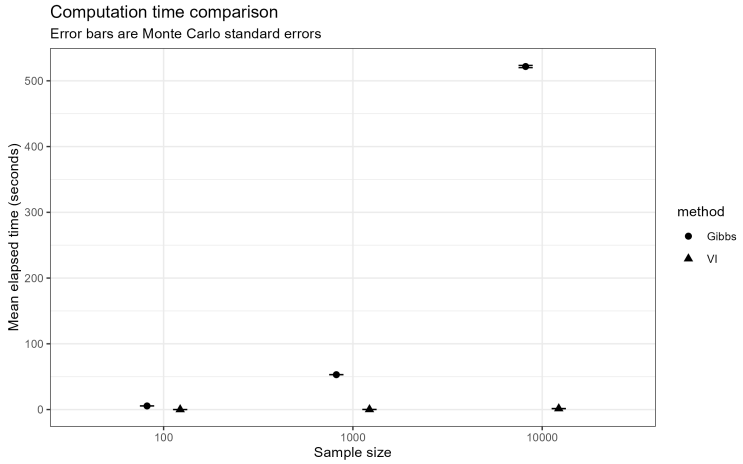
```
knitr::include_graphics(
  here::here("results", "simulation_study", "summaries", "coverage_plot.png")
)
```




### Computation time plot

Finally, the plot below compares computation time between Gibbs sampling and variational Bayes across sample sizes.

```
knitr::include_graphics(
  here::here("results", "simulation_study", "summaries", "time_plot.png")
)
```




```
par(op)
layout(1)
```

### Interpretation

This simulation study compares the statistical and computational performance of Gibbs sampling and variational Bayes in a controlled four-component mixture setting. Bias measures whether the point estimates are centered around the true component means, while coverage measures whether the uncertainty intervals achieve their nominal 95% level.

As the sample size increases, both methods are expected to produce more stable estimates, reflected in reduced bias and improved interval behavior. Computation time is also expected to increase with sample size, especially for Gibbs sampling, since it relies on repeated Monte Carlo sampling. Variational Bayes is expected to be substantially faster because it uses deterministic coordinate ascent updates.

The Monte Carlo standard error bars quantify the simulation uncertainty in the reported summaries. Smaller bars indicate that the corresponding simulation estimate is more precisely estimated from the 500 replications.

The simulation study showed the expected tradeoff between the two methods. Variational Bayes was consistently faster than the Gibbs sampler at all sample sizes, with the gap widening as $n$ increased. In terms of estimation accuracy, Gibbs sampling generally produced smaller bias and more reliable coverage, while variational Bayes was more computationally efficient but could show undercoverage or instability for some components. As the sample size increased, both methods became more stable, and the Monte Carlo standard error bars narrowed, indicating more precise simulation summaries.